CONDITIONAL STATEMENT COMPILER USING LEX AND YACC

A MINI PROJECT REPORT

submitted by

ABHINAV A (AJC24CS07)
ABIYA SUSAN CHERIAN (AJC24CS14)
AKSHAY WILSON (AJC24CS26)
AMBADI B (AJC24CS44)
ANGELINA ANNA ANTONY AJC24CS52)
ASHMY PHILO JOJI AJC24CS67)

to

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

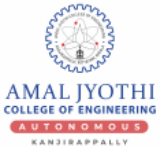
in

Computer Science and Engineering

under the guidance of

Mrs. Tintu Alphonsa Thomas

Assistant professor



Department of Computer Science and Engineering

Amal Jyothi College of Engineering (Autonomous)

Kanjirappally

October 2026

DECLARATION

We, the undersigned, hereby declare that the project report entitled “CONDITIONAL STATEMENT COMPILER USING LEX AND YACC”, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of the project work carried out by us. The work presented in this report has been prepared in good faith. Wherever the ideas or words of others have been used, the sources have been acknowledged and referenced. We declare that we have not knowingly fabricated results or misrepresented the work carried out. This report has not been submitted, in whole or in part, for the award of any other degree or diploma.

Kanjirappally

Date: 26 October 2026

ABHINAV A (Roll No. 7)

ABIYA SUSAN CHERIAN (Roll No. 14)

AKSHAY WILSON (Roll No. 26)

AMBADI B (Roll No. 44)

ANGELINA ANNA ANTONY (Roll No. 52)

ASHMY PHILO JOJI (Roll No. 67)

DEPT. OF COMPUTER SCIENCE AND ENGINEERING

AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS)

KANJIRAPPALLY

CERTIFICATE

This is to certify that the report entitled “Conditional Statement Compiler Using Lex and YACC”, submitted by Abhinav A (AJC24CS07), Abiya Susan Cherian (AJC24CS14), Akshay Wilson (AJC24CS26), Ambadi B (AJC24CS44), Angelina Anna Antony (AJC24CS52), and Ashmy Philo Joji (AJC24CS67), is a record of the project work carried out by them in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering. The report is submitted for academic evaluation.

Mrs. Tintu Alphonsa Thomas

Internal Supervisor / Project Coordinator

Dr. Juby Mathew

Head of Department, CSE

ABSTRACT

This project implements a compiler front end for a small language centered on conditional statements. The compiler processes source code through lexical analysis, syntax analysis, Abstract Syntax Tree (AST) construction, and semantic analysis. The supported statement forms include if, if-else, and nested conditional statements, together with assignments and expressions supported by the grammar. The compiler reports syntax status, displays an AST when parsing succeeds, performs semantic checks, and presents symbol-table information.

A browser-based interface built with Flask, HTML, CSS, and JavaScript allows a user to enter source code, submit it for compilation, and inspect the results. The screenshots supplied for this report show a successful if-else example, including syntax success, AST output, semantic success, and a symbol table, as well as a malformed conditional statement that produces a syntax failure. The report records only the outcomes visible in this evidence.

TABLE OF CONTENTS

Abstract 4

1**. Introduction** 6

1.1 Background 6

1.2 Problem Statement 6

1.3 Objectives 6

1.4 Scope 6

2**. Tools and Technologies** 6

3. **System Architecture** 7

4. **Context-Free Grammar (CFG)** 8

5**. Implementation** 8

5.1 Lexical Analysis 8

5.2 Syntax Analysis 9

5.3 Abstract Syntax Tree 9

5.4 Semantic Analysis 9

5.5 Integration and Web Interface 9

6**. Testing and Verification** 10

7. **Results and Discussion** 12

8. **Conclusion and Future Enhancements** 12

9. **References** 13

Appendix A: Team Members and Contributions 13

Appendix B: Project Files and Modules 14

1. INTRODUCTION

1.1 Background

Conditional statements control which statements execute according to whether a condition evaluates as true or false. The if, if-else, and nested-if forms are fundamental constructs in programming languages. A compiler front end must recognize tokens, validate grammatical structure, build an internal representation, and report invalid input.

1.2 Problem Statement

Implementing conditional statements requires coordinated lexical rules, grammar productions, syntax-error handling, an internal representation of statements and expressions, and semantic checks. This project brings these components together in a small educational compiler using Flex and YACC/Bison, with a web interface for interaction.

1.3 Objectives

- Recognize keywords, identifiers, numeric literals, operators, and punctuation.
- Define grammar rules for conditional statements, expressions, assignments, and blocks.
- Validate if, if-else, and nested conditional syntax.
- Construct and display an Abstract Syntax Tree.
- Perform basic semantic checks on conditions, operators, variables, and assignments.
- Integrate the compiler components and present results through a browser-based interface.
- Demonstrate behavior using valid and invalid source-code examples.

1.4 Scope

The project is an educational compiler front end rather than a complete general-purpose compiler. Its language includes conditional keywords, arithmetic and logical operators, relational and equality operators, identifiers, numeric literals, parentheses, braces, and semicolons. The described integration driver pre-registers x, y, and z as numeric variables and flag as a Boolean variable; the language does not include variable-declaration syntax.

2. TOOLS AND TECHNOLOGIES

Technology	**Role in the project**
Flex	Generates the lexical analyzer from lexer.l.

YACC/Bison	Generates the parser from parser.y and validates grammatical structure.
C	Implements AST operations, semantic analysis, and compiler integration.
GCC	Compiles the generated and handwritten C source files.
Make	Automates the build process through the Makefile.
Python and Flask	Provide the web server and backend request handling.
HTML, CSS, and JavaScript	Structure, style, and add client-side interaction to the web interface.
GitHub	Supports source-code storage and team collaboration, if used by the team.

3. SYSTEM ARCHITECTURE

The user enters source code in the browser-based interface. The Flask backend receives the submission and coordinates the request to the compiler executable. The compiler then performs lexical analysis using Flex, syntax analysis using YACC/Bison, AST construction, and semantic analysis. The resulting syntax status, AST, semantic status, and symbol-table information are returned for display in the web interface.

Web interface (HTML, CSS, JavaScript)

Flask backend / request handling

Compiler integration driver

Lexical analysis (Flex / lexer.l)

Syntax analysis (YACC/Bison / parser.y)

AST construction (ast.c / ast.h)

Semantic analysis and symbol table (semantic.c / semantic.h)

Results displayed in the browser

The sequence above describes the reported architecture; implementation details should match the project source code.

4. CONTEXT-FREE GRAMMAR (CFG)

The grammar describes the statement forms and expression structure accepted by the compiler. The following simplified productions summarize the grammar represented in the project documentation. The parser.y file is the authoritative source for the exact implementation.

Non-terminal	**Production**
program	→ statement_list
statement_list	→ statement_list statement | statement
statement	→ assignment | if_statement | block
if_statement	→ IF '(' condition ')' statement
if_statement	→ IF '(' condition ')' statement ELSE statement
block	→ '{' statement_list '}'
assignment	→ IDENTIFIER '=' expression ';'
expression	→ expression '||' logical_and | logical_and
logical_and	→ logical_and '&&' equality | equality
equality	→ equality '==' relational | equality '!=' relational | relational
relational	→ expression '<' expression | expression '<=' expression | expression '>' expression | expression '>=' expression | arithmetic_expression

5. IMPLEMENTATION

5.1 Lexical Analysis

The lexer recognizes if and else keywords; logical operators such as && and ||; equality and relational operators; assignment and arithmetic operators; parentheses, braces, and semicolons; numeric literals; and identifiers. Numeric literals are converted to numeric values for the parser, while identifiers are returned as identifier tokens. Whitespace is ignored as appropriate, and unsupported characters produce a lexical error.

5.2 Syntax Analysis

The YACC/Bison parser checks the input against grammar productions for statements, conditions, expressions, blocks, and conditional forms. The integration driver invokes the parser. When parsing fails, the interface reports syntax failure and indicates that an AST is unavailable; when parsing succeeds, the AST can be displayed.

5.3 Abstract Syntax Tree

The AST represents the structure of the parsed program. The project defines node types for program and block structures, if and if-else statements, assignments, binary and unary operations, identifiers, numbers, and Boolean values. The tree provides a structured view of how the input has been parsed.

5.4 Semantic Analysis

Semantic analysis uses a symbol table and type information to check conditions and expressions. The described checks include Boolean conditions for if statements, Boolean operands for logical operators, numeric operands for arithmetic and relational operators, compatible equality operands, assignment-type consistency, and direct numeric division by zero. The current language uses a predefined set of variables rather than declaration statements.

5.5 Integration and Web Interface

The compiler integration driver processes submitted source code, invokes the parser, constructs and prints the AST when available, runs semantic analysis, reports the symbol table, and releases AST memory. The browser interface is built with HTML, CSS, and JavaScript. Flask handles web requests and connects the interface to the compiler executable. The page organizes the results into syntax analysis, AST, semantic analysis, and complete compiler output panels.

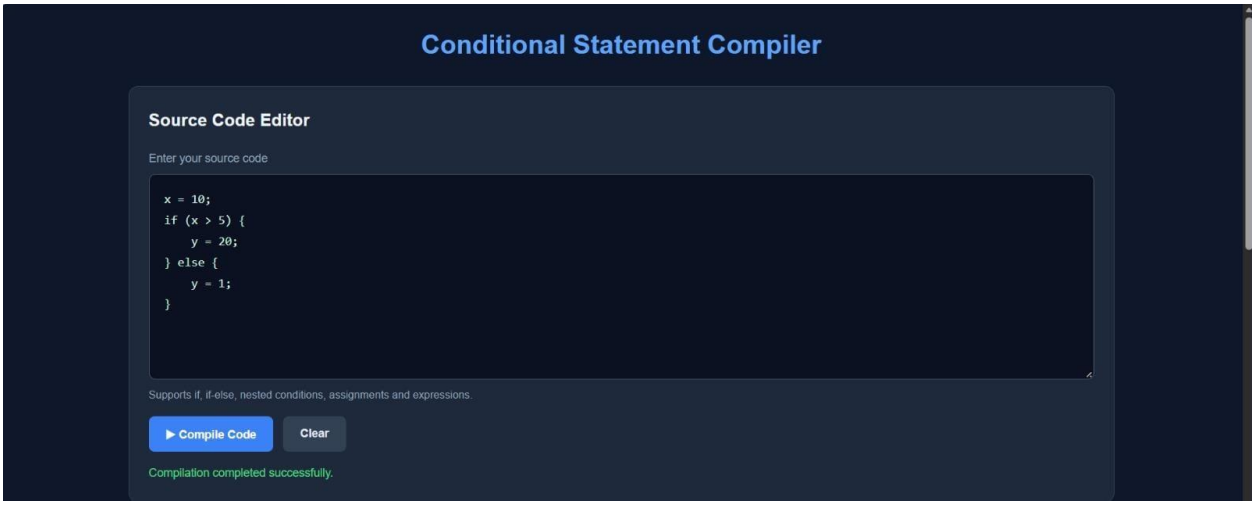


Figure 1. Web interface with a valid if-else program entered in the source-code editor.



Figure 2. Compiler results showing syntax success and the generated AST.



Figure 3. Semantic-analysis success and symbol-table output.

6. TESTING AND VERIFICATION

The supplied screenshots provide evidence for two test scenarios. The first uses a valid if-else statement and shows successful syntax analysis, an AST containing the assignment and conditional branches, successful semantic analysis, and a symbol table. The second uses an if statement with a missing closing parenthesis and shows a syntax failure. The report does not claim that additional test cases passed because no execution evidence for them was supplied.

Test	Input	Observed output	Evidence-based status
T1 – Valid if-else	x = 10; if (x > 5) { y = 20; } else { y = 1; }	Syntax SUCCESS; AST displayed; semantic analysis SUCCESS; symbol table displayed.	Pass shown in supplied screenshot
T2 – Missing closing parenthesis	if (x > 5 { y = 10; }	Syntax analysis FAILED; AST unavailable; semantic analysis not run.	Failure correctly reported in supplied screenshot

Testing note: The screenshots demonstrate these two scenarios only. A broader test suite—including nested conditions, invalid characters, missing braces, and semantic type errors—should be documented only after those inputs have actually been executed and their outputs recorded.

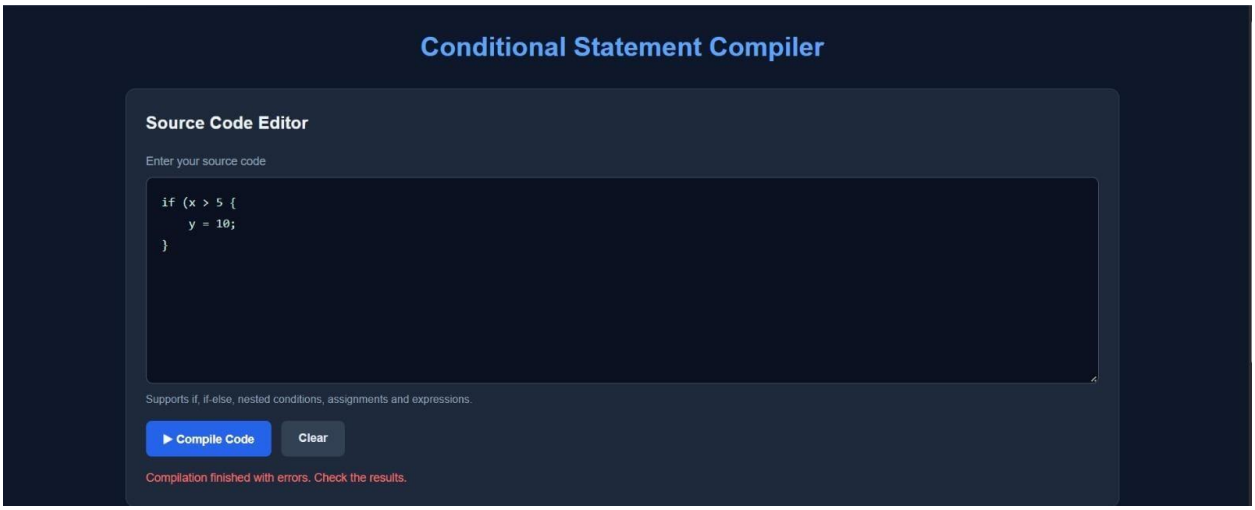


Figure 4. Web interface showing the malformed conditional statement submitted for testing.

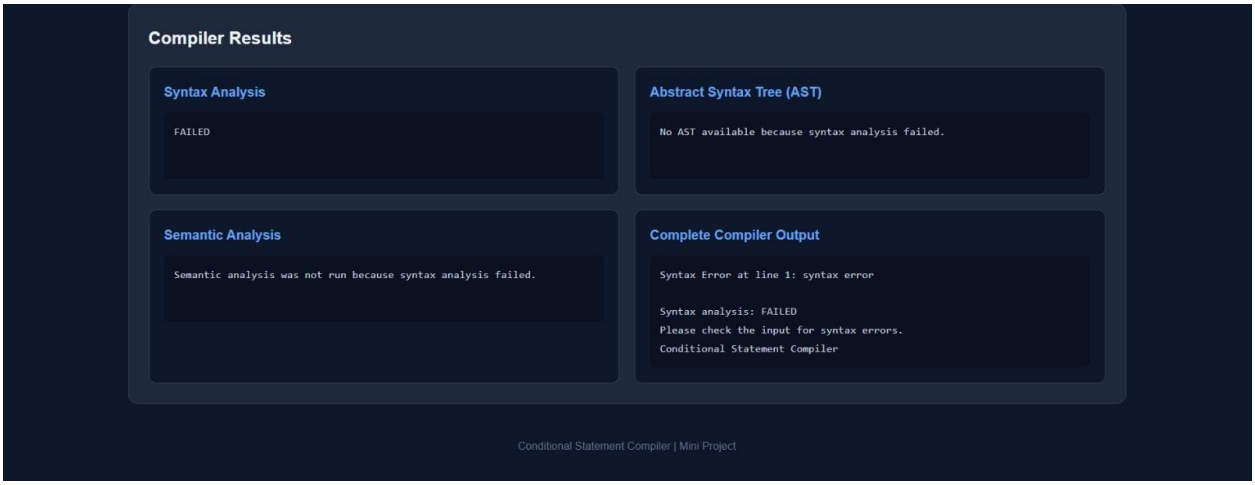


Figure 5. Compiler results for malformed input: syntax analysis failed and no AST was produced.

7. RESULTS AND DISCUSSION

The successful if-else example demonstrates that the web application can accept source code and display compiler results. The screenshot shows the parser accepting the statement, the AST representing the condition and both branches, and semantic analysis reporting no semantic errors. The symbol table lists the predefined variables and their types. The malformed-input example demonstrates that a missing closing parenthesis is detected as a syntax error, with no AST made available and semantic analysis skipped. These observations are limited to the supplied screenshots and should not be interpreted as evidence that every supported language construct has been tested.

8. CONCLUSION AND FUTURE ENHANCEMENTS

The Conditional Statement Compiler brings together the main front-end stages of a compiler: lexical analysis, syntax analysis, AST construction, and semantic analysis. The web interface provides a browser-based way to submit conditional statements and inspect the compiler's output. The supplied test evidence demonstrates successful processing of an if-else statement and detection of a syntax error caused by a missing parenthesis. Future enhancements could include variable declarations, more detailed error locations, stronger error recovery, automated regression tests, and support for additional statement types.

9. REFERENCES

1. Flex Project. The Flex Manual. https://westes.github.io/flex/manual/
2. GNU Project. GNU Bison Manual. https://www.gnu.org/software/bison/manual/
3. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
4. MDN Web Docs. HTML, CSS, and JavaScript Documentation. https://developer.mozilla.org/
5. Free Software Foundation. GCC Online Documentation. https://gcc.gnu.org/onlinedocs/

APPENDIX A: TEAM MEMBERS AND CONTRIBUTIONS

Member	**Responsibility**	**Contribution**
Ambadi B (Roll No. 44)	Lexical Analyzer and GUI Development	Developed the Flex-based lexical analyzer and Flask GUI to compile code and display syntax, AST, and semantic analysis results.
Angelina Anna Antony (Roll No. 52)	CFG and Grammar	Prepared docs/grammar.md, defined the CFG and grammar rules for conditional statements, expressions, and blocks, and documented grammar transformations and examples.
Ashmy Philo Joji (Roll No. 67)	Syntax Analyzer	Implemented the YACC/Bison parser in parser.y to validate syntax, support nested if and if-else statements, and detect syntax errors.
Abhinav A (Roll No. 7)	AST and Semantic Analysis	Developed ast.h, ast.c, semantic.h, and semantic.c for AST construction, printing, type checking, Boolean condition validation, and semantic error detection.
Akshay Wilson (Roll No. 26)	Integration and Testing	Integrated the compiler components using main.c and the Makefile. Prepared valid and invalid test cases and verified compiler functionality.
Abiya Susan Cherian (Roll No. 14)	Complete Report and PPT Preparation	Prepared the project report and presentation, organized screenshots and test results, and maintained

		README.md and project documentation.

APPENDIX B: PROJECT FILES AND MODULES

File / folder	Purpose
lexer.l	Flex lexical rules and token recognition.
docs/grammar.md	Context-free grammar and grammar examples.
parser.y	YACC/Bison grammar and syntax validation.
ast.c / ast.h	AST node definitions, construction, printing, and cleanup.
semantic.c / semantic.h	Symbol table and semantic type checks.
main.c	Compiler integration entry point and output.
Makefile	Build automation for Flex, Bison, and GCC.
tests/	Test inputs and test evidence, if present in the project repository.
Flask application / templates / static assets	Backend routes and HTML, CSS, and JavaScript files; exact filenames depend on the repository layout.